Course Summary: Frontend Engineering Onboarding: Advanced React & Modern Standards

Module 1: Development Environment & Architecture

Lesson 1: Monorepo Navigation & Architecture

Key Takeaways:

• The monorepo separates deployable applications in /apps from reusable libraries in /packages.
• Turborepo is used to orchestrate builds, linting, and testing across workspaces.

Lesson 2: Local Environment Setup & Tooling

Key Takeaways:

• Always use the designated Node.js version and pnpm for package management.
• Husky pre-commit hooks enforce code formatting and linting automatically.

Lesson 3: Styling Standards & Design System Integration

Key Takeaways:

• Tailwind CSS utility classes must align with our shared design tokens.
• Avoid arbitrary values in Tailwind; use semantic tokens to maintain design consistency.

Flashcards:

Q: Turborepo
A: The build system used to orchestrate tasks, caching, and workspace execution in our monorepo.

Q: pnpm
A: Our fast, disk-efficient package manager that uses a content-addressable store and hard links.

Q: Design Tokens
A: Visual design atoms (colors, spacing, typography) defined in our shared package and integrated with Tailwind CSS.

Module 2: Advanced React Patterns & State Management

Lesson 1: Component Design & Composition Patterns

Key Takeaways:

• Use compound components to manage implicit state in complex UI controls.
• Composition reduces prop drilling and enhances component flexibility.

Lesson 2: Custom Hooks & Logic Extraction

Key Takeaways:

• Custom hooks isolate business logic, leaving components focused on rendering.
• Always implement cleanup functions within hooks to prevent memory leaks.

Lesson 3: State Management: Local vs. Global Solutions

Key Takeaways:

• Keep state as local as possible to optimize rendering performance.
• Use Zustand for global client-side state, and reserve React Context for low-frequency updates.

Flashcards:

Q: Compound Components
A: A design pattern where components work together to manage state implicitly, avoiding prop drilling.

Q: Zustand
A: A lightweight, fast, and selector-based state management library used for global client-side state.

Q: Custom Hooks
A: Functions starting with 'use' that allow you to extract and share stateful logic across components.

Module 3: Data Fetching & API Integration

Lesson 1: Data Fetching & Caching Strategies

Key Takeaways:

• TanStack Query is our standard for managing, caching, and syncing server state.
• Optimistic updates improve perceived performance during data mutations.

Lesson 2: API Integration & Error Handling

Key Takeaways:

• Axios interceptors handle global request and response concerns.
• Error Boundaries isolate runtime failures and preserve application stability.

Lesson 3: TypeScript Typing for API Contracts

Key Takeaways:

• Generate TypeScript types from backend schemas to ensure contract alignment.
• Use Zod to validate external API payloads at the application boundary.

Flashcards:

Q: TanStack Query
A: Our standard library for fetching, caching, synchronizing, and updating server state in React.

Q: Zod
A: A TypeScript-first schema declaration and validation library used to validate API payloads at runtime.

Q: Error Boundary
A: A React component that catches JavaScript errors anywhere in its child component tree and displays a fallback UI.

Module 4: Testing Paradigms & Quality Assurance

Lesson 1: Unit Testing with Jest

Key Takeaways:

• Jest is our primary runner for unit testing pure logic and utilities.
• Mock external modules to keep unit tests fast and isolated.

Lesson 2: Integration Testing with React Testing Library

Key Takeaways:

• React Testing Library tests component behavior, not implementation details.
• Query elements by accessible roles to ensure tests align with real-world usage.

Lesson 3: Testing for Accessibility (a11y)

Key Takeaways:

• Automate accessibility testing using jest-axe in your test suites.
• Semantic HTML and robust focus management are essential for keyboard navigation.

Flashcards:

Q: React Testing Library
A: A testing utility focused on testing React components from the user's perspective without relying on implementation details.

Q: jest-axe
A: A library used to run automated accessibility (a11y) audits within Jest tests.

Q: getByRole
A: The preferred React Testing Library query that searches for elements within the accessibility tree.

Module 5: Performance Tuning & Code Delivery

Lesson 1: Performance Profiling & Optimization

Key Takeaways:

• Profile components before applying memoization optimizations.
• Use React.lazy and Suspense to split code and reduce initial bundle sizes.

Lesson 2: Code Review Standards & Best Practices

Key Takeaways:

• Small, atomic Pull Requests speed up reviews and reduce integration bugs.
• Conventional Commits are required to maintain a clean, automated changelog.

Lesson 3: CI/CD Pipelines & Deployment Workflows

Key Takeaways:

• CI pipelines enforce quality checks automatically before code can be merged.
• Preview deployments facilitate collaborative QA and design reviews.

Flashcards:

Q: React.lazy
A: A function that lets you render a dynamic import as a regular component, enabling code splitting.

Q: Conventional Commits
A: A specification for adding human and machine-readable meaning to commit messages.

Q: Preview Deployment
A: An isolated, ephemeral deployment generated automatically for each Pull Request to facilitate QA and review.